AstroVed Platform — Project Documentation

Document version: 1.0
Stack: Next.js (App Router), React, TypeScript
Generated for: Internal / company reference

---

1. Executive summary

This repository is a **full-stack spiritual services web application** branded around AstroVed-style experiences. It combines a **public marketing and booking surface** (temples, pujas, chadhava, panchang, library, store, astrologers, astro tools) with **user authentication**, **profile**, **bookings**, **payments**, and a **protected admin dashboard** for content and catalog management.

The frontend is built with **Next.js 16** using the **App Router**. Server routes under `src/app/api/` implement REST-style handlers. **MongoDB** is used for persistence (users, bookings, and admin-managed content). **JWT** sessions are issued via the **jose** library and stored in **HTTP-only cookies** (`userToken` for end users, `adminToken` for administrators).

---

2. Project overview

2.1 Purpose

- Present temples, pujas, chadhava offerings, and related spiritual content.
- Allow users to explore **panchang**, **astro tools** (e.g. janma rashi, lagna, nakshatra), **library** content, and **store** items.
- Support **consultation** and **booking** flows toward **payment**.
- Provide **sign up / sign in** (email + password, phone OTP, WhatsApp OTP) and a **user profile**.
- Enable **staff** to manage content through `/admin` (separate login, JWT cookie).

2.2 High-level architecture

Layer	Responsibility
UI (React)	Pages in `src/app/` , reusable components in `src/components/`
API routes	`src/app/api/**/route.ts` — auth, bookings, admin, panchang helpers, etc.
Legacy APIs	`src/pages/api/*` — older Pages Router endpoints still in use
Data	MongoDB via `src/lib/mongodb.ts`
Auth	`bcryptjs` (password hash), `jose` (JWT), cookies

Layer	Responsibility
Styling	Tailwind CSS v4, global tokens in `src/app/globals.css`

3. Technology stack

3.1 Core

Tool	Version (approx.)	Role
Next.js	16.2.4	Framework, SSR/SSG, API routes
React	19.2.4	UI
TypeScript	^5	Type safety
Tailwind CSS	4.2.x	Utility-first styling
PostCSS / Autoprefixer	—	CSS pipeline

3.2 Backend & security

Tool	Role
MongoDB (`mongodb` driver)	Database
bcryptjs	Password hashing (signup) and comparison (login)
jose	JWT sign/verify for `userToken` and `adminToken`

3.3 UI & UX libraries

Tool	Role
Framer Motion	Animations
@heroicons/react	Icons
react-international-phone	Country flags + dial codes for phone/WhatsApp inputs

3.4 Tooling

Tool	Role
ESLint	Linting (`eslint-config-next`)

3.5 External integrations (examples)

- Remote images configured in `next.config.ts` (Unsplash, CDNs, partner hosts).
- Optional **OTP provider** hook via env (see `src/lib/server/authOtpStore.ts` — `AUTH_OTP_PROVIDER_URL` in production).

---

4. Repository structure (concise)

```
src/
  app/                # App Router: pages + layouts
    api/              # Route handlers (auth, bookings, admin, tools, …)
    auth/             # login, signup, forgot-password, otp
    admin/            # Dashboard (protected)
    …                 # public routes: temples, puja, chadhava, panchang, etc.
  components/         # Shared UI (layout, booking, auth, admin, …)
  contexts/           # e.g. language
  hooks/              # Custom hooks
  lib/                # mongodb, countries, server OTP store, etc.
  services/           # e.g. authService — centralized client auth calls
  types/              # Shared TypeScript types (auth, bookings, …)
  pages/api/          # Legacy API routes (Pages Router)
middleware.ts         # Admin + selective user route protection
public/               # Static assets
```

---

5. User flows

5.1 Discovery & content

1. User lands on **home** (`/`).
2. Navigates **Puja**, **Chadhava**, **Temples**, **Panchang**, **Library**, **Store**, **Astro tools**, **Astrologers**, **About**, **Contact**, etc.
3. Opens **dynamic detail** routes, e.g. `puja/[slug]` , `chadhava/[slug]` , `temples/[slug]` , `astrologers/[id]` .

5.2 Registration

1. User opens `/auth/signup`.
2. Enters **full name**, **email**, **country** (searchable), **mobile** (international phone input).
3. Optionally checks **separate WhatsApp** and enters **WhatsApp number**; otherwise WhatsApp is stored as the same national number as mobile (server-side).
4. Sets **password** + **confirm password** (strength hints; show/hide toggles).
5. `POST /api/auth/signup` validates input, hashes password with **bcrypt**, stores user in MongoDB (no plaintext password).
6. Redirect to `/auth/login` (manual login after registration).

5.3 Login

1. User opens `/auth/login`.
2. **Tabs:** Email + password | Phone OTP | WhatsApp OTP.
3. **Email:** `POST /api/auth/login` — compares password to `passwordHash`, sets `userToken` cookie (JWT, HTTP-only).
4. **Phone / WhatsApp:** `POST /api/auth/otp/send` — OTP stored server-side (dev default code documented in code); user redirected to `/auth/otp` to enter 6 digits, **resend** with cooldown, **verify** via `POST /api/auth/otp/verify`, then `userToken` set.
5. Optional `callbackUrl` query param returns user to the intended page after login.

5.4 Profile & dashboard

- `/profile`: Reads `GET /api/auth/me` (JWT from cookie); shows name, contact, country, etc.
- `/dashboard`: Main logged-in style home / hub (content may vary).

5.5 Booking & payment

- Booking entry points: e.g. `/booking`, `/bookings/puja`, `/bookings/chadhava`.
- `/payment`: Middleware treats payment-related paths as **user-protected** — requires valid `userToken` or redirect to login with `callbackUrl`.

5.6 Admin

1. User opens `/admin/login`.
2. Email may be **prefilled** from `GET /api/admin/login-prefill` (reads `ADMIN_EMAIL` only; **password is never returned**).
3. `POST /api/admin/login` checks credentials against `ADMIN_EMAIL` / `ADMIN_PASSWORD`; sets `adminToken` cookie.
4. `/admin/*` (except login) requires valid `adminToken`; middleware redirects otherwise.

---

6. Authentication & security notes

- **Passwords:** Hashed with **bcrypt** on signup; login uses **bcrypt.compare** against `passwordHash`. Do not log or store plaintext passwords.
- **Sessions:** JWT in **HTTP-only** cookies; configure `JWT_SECRET` in production (avoid default fallback).
- **Admin:** Separate cookie and env vars `ADMIN_EMAIL`, `ADMIN_PASSWORD`.
- **OTP:** In-memory store in development; production should wire `AUTH_OTP_PROVIDER_URL` (and related env) for real SMS/WhatsApp delivery.

---

7. Environment variables (reference)

Variable	Typical use
`MONGODB_URI`	MongoDB connection string
`JWT_SECRET`	Signing key for user and admin JWTs
`ADMIN_EMAIL` / `ADMIN_PASSWORD`	Admin dashboard login
`AUTH_OTP_PROVIDER_URL`	Optional OTP delivery service
`AUTH_OTP_PROVIDER_TOKEN`	Optional bearer token for OTP provider
`NODE_ENV`	`production` affects cookie `secure` flag

(Exact set may grow; check `.env.local` and route handlers.)

---

8. API surface (non-exhaustive)

Area	Examples
Auth	`/api/auth/signup`, `/api/auth/login`, `/api/auth/logout`, `/api/auth/me`, `/api/auth/otp/send`, `/api/auth/otp/resend`, `/api/auth/otp/verify`
Admin	`/api/admin/login`, `/api/admin/login-prefill`, `/api/admin/logout`, `/api/admin/content`, `/api/admin/stats`, …
Bookings	`/api/bookings/create`, `/api/bookings/me`
Content	Various routes under `src/app/api/` and `src/pages/api/` for pujas, temples, reviews, panchang data, etc.

9. Frontend conventions

- **Auth API calls** are centralized in `src/services/authService.ts` (and thin re-exports in `src/lib/api/auth.ts` where applicable).
- **Types** for auth payloads/responses live in `src/types/auth.ts`.
- **International phone** styling is in `src/app/globals.css` (`.astroved-phone`).

10. Build & run

```
npm install
npm run dev      # development
npm run build    # production build
npm run start    # production server
npm run lint     # ESLint
```

11. Known considerations

- **Middleware matcher** includes several path prefixes; business rules inside middleware currently emphasize `/admin` protection and `/payment` (and related) user JWT checks — verify as routes evolve.
- **MongoDB** must be reachable for signup/login and data APIs; build-time static generation may log connection warnings if the DB is offline.
- **Next.js 16** may deprecate older conventions (e.g. middleware messaging in build output); follow official upgrade guides.

12. Document maintenance

Update this file when major features, env vars, or flows change. Regenerate the PDF from this Markdown using your preferred toolchain (e.g. `md-to-pdf` , print-to-PDF from HTML, or CI artifact).

End of document.